

22518 Software Testing

Unit-IV Defect Management

4.1. Defect Classification, Defect Management Process.

Defect Classification

Defect classification involves categorizing the defects discovered during the software testing process based on various factors. This helps in understanding the defect trends, prioritizing the defects for fixing, and improving the software development and testing processes.

Common ways to classify defects include:

1. **Severity:** This indicates the impact of the defect on the functionality of the software. The typical classifications based on severity are:
 - Critical: The defect causes a complete shutdown of the system or major loss of service.
 - High: The defect causes significant functional loss, but the system can still operate.
 - Medium: The defect causes some functional loss, which is tolerable and the system can still function.
 - Low: The defect has minimal impact on the system's functionality.
2. **Priority:** This refers to the order in which the defects should be fixed. The typical classifications based on priority are:
 - Urgent: The defect needs to be fixed immediately.
 - High: The defect needs to be fixed as soon as possible.
 - Medium: The defect can be fixed in the normal course of development and testing.
 - Low: The defect can be deferred until all other defects have been fixed.
3. **Defect Type:** This refers to the nature of the defect. Some common defect types include:
 - Functional: The defect is related to the functionality of the software.
 - Performance: The defect is related to the performance of the software (speed, response time, resource usage, etc.).
 - Usability: The defect affects the usability of the software.
 - Security: The defect affects the security of the software.
4. **Defect Origin:** This refers to the phase of the software development lifecycle in which the defect was introduced. It can help identify which phases of the lifecycle need improvement.

By categorizing defects in this manner, teams can better manage their resources and prioritize their testing and development efforts.

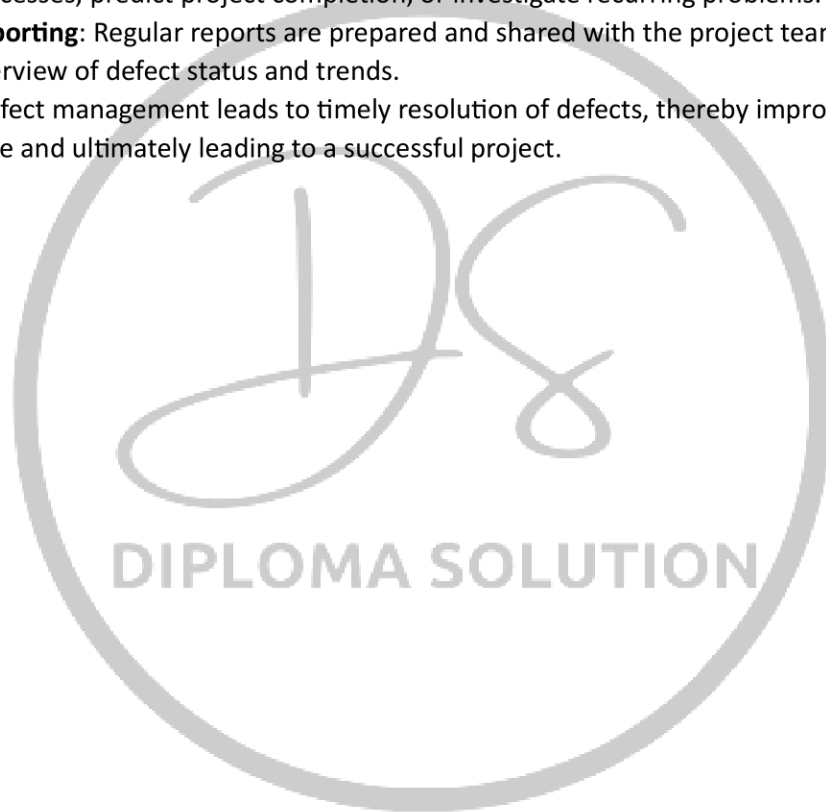
Defect Management Process.

Defect management is a critical aspect of software development and testing. It involves the process of detecting, recording, and resolving defects. Here is a typical defect management process:

1. **Defect Identification:** This is the first step where testers identify a defect during the testing phase. They determine that the software is not performing as expected.
2. **Defect Logging:** Once a defect is identified, it is logged in a defect tracking system. Important details like the description of the defect, the steps to reproduce it, severity, priority, module where it was found, etc., are recorded.

3. **Defect Review:** The logged defect is then reviewed to confirm its validity. Sometimes, it may not be a defect but a misunderstanding of the requirement by the tester or it could be a duplicate of an existing defect.
4. **Defect Assignment:** Once the defect is reviewed and confirmed, it is assigned to a developer to fix. The developer gets the details from the defect tracking system.
5. **Defect Fixing:** The developer works on the defect, modifies the code and checks it back into the source code repository.
6. **Re-testing:** After the code is fixed, it goes back to testing where the tester re-tests to ensure that the original defect is indeed fixed and has not resulted in other parts of the software breaking.
7. **Defect Closure:** If the tester finds the solution satisfactory, the defect is closed.
8. **Defect Tracking:** Throughout the lifecycle of the defect, it is tracked from discovery to closure. Tracking helps to provide a history of defects which can be used to improve processes, predict project completion, or investigate recurring problems.
9. **Reporting:** Regular reports are prepared and shared with the project team to provide an overview of defect status and trends.

Effective defect management leads to timely resolution of defects, thereby improving the quality of the software and ultimately leading to a successful project.



4.2. Defect Life Cycle, Defect Template

Defect Life Cycle

The Defect Life Cycle, also known as the Bug Life Cycle, is the journey of a defect from its discovery to its closure. This is a key aspect of defect management and understanding it allows teams to manage defects effectively. The stages of the defect life cycle are typically as follows:

1. **New:** This is the first stage of the defect life cycle. When a defect is discovered and logged for the first time, its status is marked as new.
2. **Assigned:** Once the defect has been logged, it is evaluated by a project lead or manager. If the defect is valid, it is assigned to a developer or a team for fixing.
3. **Open:** At this stage, the developer starts analyzing and working on the defect fix.
4. **Fixed:** The developer makes necessary code changes and verifies the fix. The status of the defect is then changed to fixed. The solution needs to be retested, and the defect is marked for verification.
5. **Test:** During this stage, the tester retests the code to ensure that the defect has been fixed and is not causing further issues.
6. **Verified:** If the tester finds that the defect has been successfully fixed, then its status is changed to verified.
7. **Reopen:** If the defect still exists even after the fix, the tester changes the status to reopen. The defect goes through the life cycle again.
8. **Closed:** If the tester feels that the defect no longer exists, the status of the defect is changed to closed. This is the final stage of the defect life cycle.
9. **Duplicate:** If the defect is reported twice, one defect status is changed to duplicate.
10. **Deferred:** If the defect is expected to be fixed in the next releases, the status is changed to deferred.
11. **Rejected:** If the developer feels that the defect is not genuine, they can mark the status as rejected.
12. **Not a bug:** If it is as per the functionality of the application, then the bug can be marked as not a bug.

Understanding the defect life cycle helps the software team to track and control the defects throughout the development process, leading to better software quality.

Defect Template

A defect template is a document used to capture and track details about a discovered defect during software testing. It allows for standardized reporting and can streamline the process of communication and resolution between testers and developers. While the specific fields can vary depending on the team or the defect tracking tool being used, here is a common set of details that are usually captured in a defect template:

1. **Defect ID:** A unique identifier for each defect.
2. **Title:** A short, clear description of the issue.
3. **Description:** A detailed account of the defect including what is expected and what is happening instead.
4. **Steps to Reproduce:** Clear, step-by-step instructions to reproduce the defect. This should include any necessary setup and the exact inputs used.
5. **Severity:** The level of impact the defect has on the system. Levels can include critical, major, minor, and trivial.
6. **Priority:** The level of urgency for fixing the defect. Levels can include high, medium, and low.

7. **Status:** The current status of the defect in the defect lifecycle, such as new, assigned, open, fixed, retest, verified, closed, etc.
8. **Detected By:** Name/ID of the tester who found the defect.
9. **Assigned To:** Name/ID of the developer or team who is responsible for fixing the defect.
10. **Date Raised:** The date when the defect was first discovered and reported.
11. **Environment:** Details about the environment where the defect was found, such as software version, hardware, etc.
12. **Screenshot/Attachment:** If applicable, screenshots or other attachments that show the defect.
13. **Resolution/Fix Description:** Once fixed, details about what was done to address the defect.
14. **Resolved Date:** The date when the defect was fixed.
15. **Comments/Notes:** Any additional information or comments about the defect.

By using a defect template, a team can ensure that all necessary information about each defect is captured and communicated effectively, making it easier to understand and resolve the issues.



4.3. Estimate Expected Impact of a Defect, Techniques for Finding Defects, Reporting a Defect.

Estimate Expected Impact of a Defect

Estimating the expected impact of a defect is a crucial part of the software testing process. It helps in prioritizing the defects and in deciding the corrective action required. Here are some factors that are considered while estimating the expected impact of a defect:

1. **Severity of the Defect:** This refers to the degree of impact the defect has on the system's functionality. Severe defects might cause the system to fail or crash, or could affect critical functionalities. Lower severity defects might impact non-critical functionalities or cause minor inconveniences.
2. **Frequency of Occurrence:** How often the defect occurs can also influence its impact. A defect that occurs frequently can have a larger impact compared to a defect that occurs rarely, even if the individual impact of each occurrence is low.
3. **User Impact:** If the defect affects a feature that is frequently used by many users, the overall user impact can be high. If the defect only affects a few users or a feature that is rarely used, the user impact may be lower.
4. **Business Impact:** The impact on the business operations and objectives is also considered. A defect that affects a critical business process or objective can have a high impact.
5. **Data Impact:** Defects that can lead to loss of data or corruption of data can have a very high impact, given the importance of data in any software system.
6. **Security Impact:** Defects that expose security vulnerabilities can have a very high impact, as they can potentially lead to breaches and unauthorized access.

After considering these factors, an expected impact can be estimated for the defect. This helps the team to prioritize which defects to fix first and to allocate resources effectively.

Techniques for Finding Defects

There are several techniques for finding defects in software systems. These techniques can be broadly categorized into static and dynamic testing techniques:

Static Testing Techniques:

1. **Reviews:** This can be peer reviews, walkthroughs, or formal inspections. Reviews are done on the documentation including requirements, design documents, and source code.
2. **Static Analysis:** This is usually carried out by tools and includes techniques like data flow analysis, control flow analysis, cyclomatic complexity measurement, etc. It helps to find issues like unreachable code, syntax violations, memory leaks, etc.

Dynamic Testing Techniques:

1. **Unit Testing:** This is typically done by developers. It involves testing individual modules or components of the software to ensure they are working correctly.
2. **Integration Testing:** This involves testing the interaction between different modules of the software.
3. **System Testing:** This is a high-level testing where the entire system is tested to ensure it meets the specified requirements.
4. **Acceptance Testing:** This is usually the final testing done before the software is released. It involves testing the system in the real environment by the intended audience.

Manual and Automated Testing:

1. **Manual Testing:** This involves testers manually executing test cases and observing the results.

2. **Automated Testing:** This involves using automated tools to execute test cases. Automated testing is typically used for regression testing, performance testing, load testing, etc.

Experience-based Techniques:

1. **Exploratory Testing:** This involves testers exploring the software based on their experience, knowledge and intuition.
2. **Error Guessing:** Here the tester guesses the most probable areas where defects can occur based on their past experience.

Defect Clustering and Pareto Analysis: In defect clustering, testing effort is concentrated on areas of the software where defects are most likely to be clustered. Pareto analysis is a statistical technique used in decision-making to identify a set of priorities.

Choosing the appropriate technique depends on several factors like the type and complexity of the software, the development lifecycle, project timeline, and resources available.

Reporting a Defect

Reporting a defect is a crucial part of the testing process. It involves detailing the defect in such a way that developers can reproduce and fix it. To effectively report a defect, it's essential to provide as much detail as possible.

Here are the steps involved in reporting a defect:

1. **Identify Defect:** During testing, if the software doesn't behave as expected, it may be due to a defect.
2. **Record Defect:** As soon as you find a defect, record it. It's crucial to document the defect at the right time to avoid forgetting any details.
3. **Defect Details:** Include as much information as possible. Here are some essential fields in a defect report:
 - **Defect ID:** A unique identifier for the defect.
 - **Title:** A concise summary of the defect.
 - **Description:** A detailed explanation of the defect.
 - **Steps to Reproduce:** Clear instructions on how to reproduce the defect.
 - **Expected Result:** What the system should do if it were working correctly.
 - **Actual Result:** What the system actually does, which is causing the issue.
 - **Severity:** The level of impact on the system (Critical, High, Medium, Low).
 - **Priority:** The order in which the defect should be fixed (High, Medium, Low).
 - **Attachments:** Screenshots, logs, or any other files that support your findings.
4. **Submit Report:** After documenting the defect, submit the report using a bug tracking tool. This could be software like Jira, Bugzilla, or any other tool your team uses.
5. **Re-testing & Closure:** Once the developer fixes the defect, re-test the functionality to ensure the problem has been resolved. If it's fixed, close the defect in the defect tracking tool. If the defect still persists, reopen it and notify the development team.

Remember, the aim of a defect report is to enable another person to reproduce the defect, understand what's wrong, and ultimately fix it. Clear and complete reports make the process smoother for everyone involved.



"DIPLOMA SOLUTION"

Connect with Us...



+91 8108332570



MSBTE Diploma Solution



www.diplomasolution.com

